

TFI PROGRAMACIÓN II

Aplicación Java con Relación 1→1 (Empleado → Legajo), DAO y

MySQL

Aplicación Java que implementa una relación 1→1 entre Empleado y Legajo usando JDBC, patrón DAO y MySQL. El trabajo incluye diseño en capas, validaciones, transacciones y pruebas de funcionamiento.



INTEGRANTES:

 ALEJO TOMAS OLIVA COCA

 CAMILA CASTAÑO

 CHRISTIAN FERNANDO ORMACHEA

 ENZO MEDINA

Coordinador: Oscar Londero

Grupo 14 - Programación 2025

Trabajo Final Integrador – Programación II

Aplicación Java con Relación 1→1 — Empleado → Legajo

Integrantes:

- Integrante A: Diseño UML, Base de Datos, Conexión
- Integrante B: Entidades y DAO
- Integrante C: Servicios, Validaciones y Transacciones
- Integrante D: Menú, Documentación, GitHub y Coordinación General

| 1. Introducción

La finalidad de este trabajo integrador es desarrollar una aplicación completa en Java que utilice JDBC, el patrón DAO, servicios con validaciones y manejo de transacciones, y una relación 1→1 unidireccional entre dos entidades:

Empleado y Legajo.

El proyecto busca aplicar conceptos fundamentales de la programación orientada a objetos, diseño por capas, acceso a datos, integridad referencial y trabajo colaborativo.

Además del desarrollo técnico, también se requiere producir un informe que explique las decisiones tomadas, la arquitectura utilizada, las operaciones CRUD implementadas y el funcionamiento de las transacciones (commit y rollback).

Este documento presenta el diseño, la estructura interna del código, las decisiones técnicas de cada integrante, la base de datos utilizada, las reglas de negocio aplicadas y las pruebas de funcionamiento.

| 2. Dominio elegido y justificación

El dominio elegido para este trabajo fue **Empleado → Legajo**, ya que representa una estructura común en sistemas administrativos reales. Cada empleado cuenta con un único legajo que almacena información interna vinculada a su historial laboral.

Este dominio permite trabajar cómodamente con una relación **1 a 1**, que fue uno de los principales requisitos del trabajo integrador. En lugar de utilizar un dominio artificial o demasiado complejo, se eligió uno familiar y aplicable a situaciones reales de empresas u organizaciones.

Las razones principales por las que se eligió este dominio fueron:

- ▶ Permite modelar una relación 1➡1 de manera clara y lógica.
- ▶ Es un caso frecuente en bases de datos reales.
- ▶ Favorece la aplicación de validaciones y reglas de negocio.
- ▶ Se adapta bien al enfoque en capas del proyecto.
- ▶ Es sencillo de extender si en el futuro se quisieran agregar más entidades.

De esta manera, el dominio Empleado–Legajo permitió cumplir plenamente con los objetivos pedagógicos del trabajo, manteniendo coherencia entre la base de datos, el código y la estructura general del proyecto.

| 3. Diseño por capas

El proyecto se desarrolló utilizando una arquitectura dividida en capas. Esta estructura ayuda a mantener un código ordenado, fácil de mantener y con responsabilidades bien separadas.

3.1 Capa Entities (o Modelo)

Esta capa contiene las clases que representan los datos principales del sistema.

En nuestro caso:

- Empleado
- Legajo
- EstadoLegajo
- Entidades

Estas clases reflejan directamente la estructura del dominio: un empleado con su información personal y un legajo asociado que contiene datos laborales específicos.

Cada clase incluye :

- atributos propios
- constructores
- getters y setters
- un campo “eliminado” para la baja lógica

- métodos como `toString()` para mostrar información

3.2 Capa DAO (Data Access Object)

Los DAO se encargan de interactuar directamente con la base de datos usando JDBC.

Aquí se implementan los siguientes métodos:

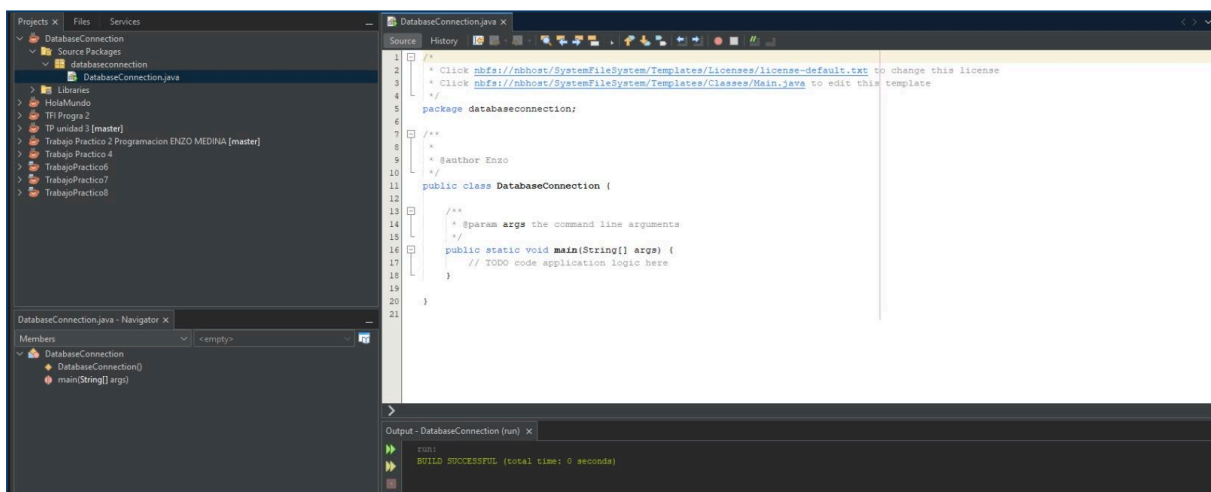
- crear
- leer
- leerTodos
- actualizar
- eliminar (baja lógica)

Los DAO del proyecto fueron:

- EmpleadoDao
- LegajoDao
- GenericDao
- Dao (interfaz base)

Todos utilizan `PreparedStatement`, lo cual evita inyecciones SQL y mejora la seguridad del sistema.

Además, aceptan una `Connection` externa, lo cual fue esencial para poder manejar transacciones desde la capa service.



Esta imagen muestra la clase `DatabaseConnection.java`, encargada de manejar la conexión JDBC entre Java y MySQL. Desde esta clase se obtiene la `Connection` utilizada por todos los DAO para ejecutar operaciones en la base de datos. Forma parte del trabajo del Integrante A (diseño y conexión a la base).

3.3 Capa Service (Reglas de negocio y Transacciones)

Esta capa se encarga de la lógica más importante:

- Validar datos antes de guardar
- Controlar las restricciones de la relación 1→1
- Manejar errores y excepciones
- Ejecutar las transacciones (commit / rollback)

Aquí se encuentran:

- EmpleadoService
- LegajoService
- GenericService
- BusinessException

Los servicios coordinan las operaciones de los DAO y garantizan que no se viole la relación uno a uno (por ejemplo, que un empleado no tenga dos legajos, o que un legajo no se asocie a varias personas).

```
1  package service;
2
3  import java.util.List;
4
5  public interface GenericService<T> {
6
7      T insertar(T entity) throws Exception;
8
9      T actualizar(T entity) throws Exception;
10
11     void eliminar(Long id) throws Exception; // baja lógica
12
13     T getById(Long id) throws Exception;
14
15     List<T> getAll() throws Exception;
16 }
17
```

***Interfaz GenericService utilizada como base de la capa Service.
Define las operaciones CRUD genéricas que deben implementar
EmpleadoService y LegajoService, permitiendo mantener una estructura
homogénea y estandarizada en la lógica de negocio***

3.4 Capa Main (Menú de Consola)

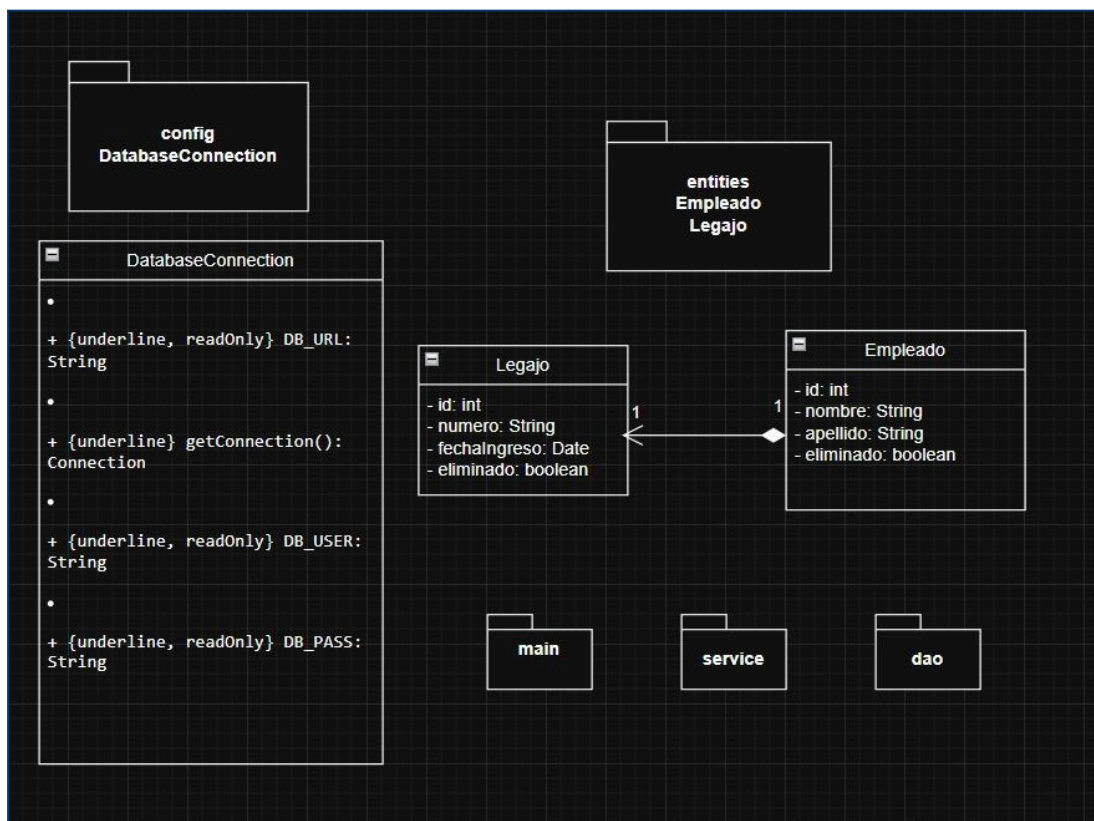
Esta capa es la que el usuario final ve.

En ella se implementa un menú que permite:

- Crear empleados
- Crear legajos
- Asociarlos
- Buscar por ID o DNI
- Listar empleados o legajos
- Actualizar datos
- Eliminar lógicamente

El objetivo de esta capa es permitir que cualquier persona pueda usar el sistema sin necesidad de conocer el código interno.

| 4. Modelo UML



Se representa la relación 1→1 unidireccional desde Empleado → Legajo, con claves, tipos y métodos principales.

| 5. Base de Datos y Scripts SQL (detalle técnico real)

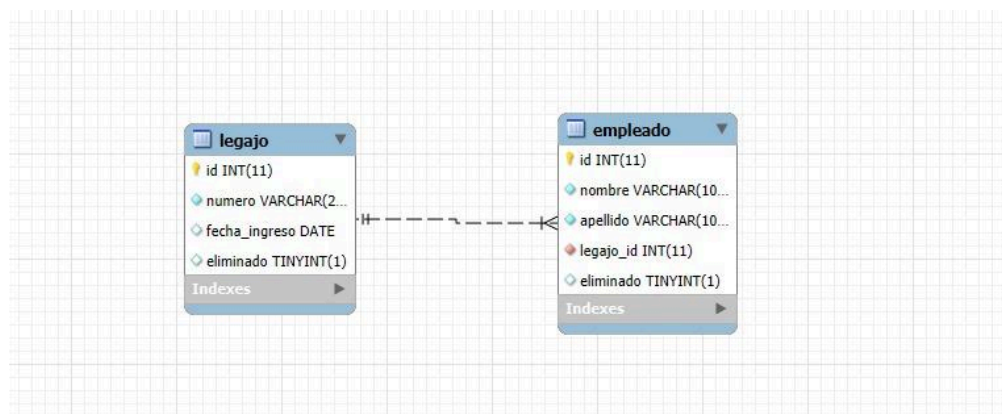
La base de datos se encuentra definida en dos archivos SQL provistos por el Integrante A:

1_structure.sql (creación de tablas y relaciones)

2_data.sql (datos iniciales de prueba)

Ambos scripts pertenecen al proyecto *TFI_Programacion2* y permiten recrear la base desde cero de forma reproducible, cumpliendo uno de los requisitos principales de la consigna.

Diagrama Relacional (DER) de la Base de Datos



“Diagrama relacional que muestra la estructura de la base de datos utilizada en el proyecto.

Se observa la relación 1→1 entre Empleado (A) y Legajo (B), donde empleado contiene la clave foránea ‘legajo_id’. Esto asegura que cada empleado tiene un único legajo asociado.”

5.1 Creación de la base de datos — 1_structure.sql

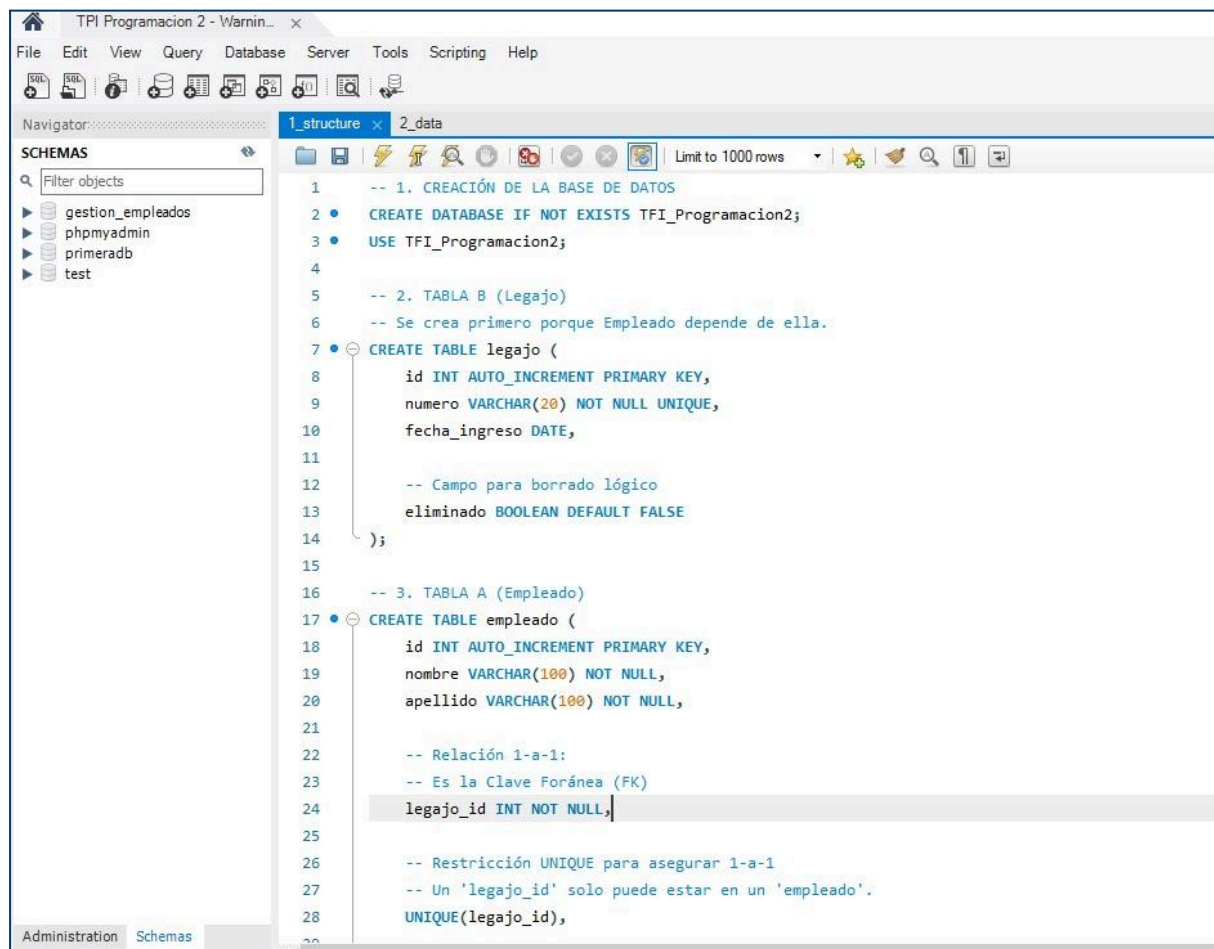
El archivo comienza creando la base de datos:

```
CREATE DATABASE IF NOT EXISTS TFI_Programacion2;
```

```
USE TFI_Programacion2;
```

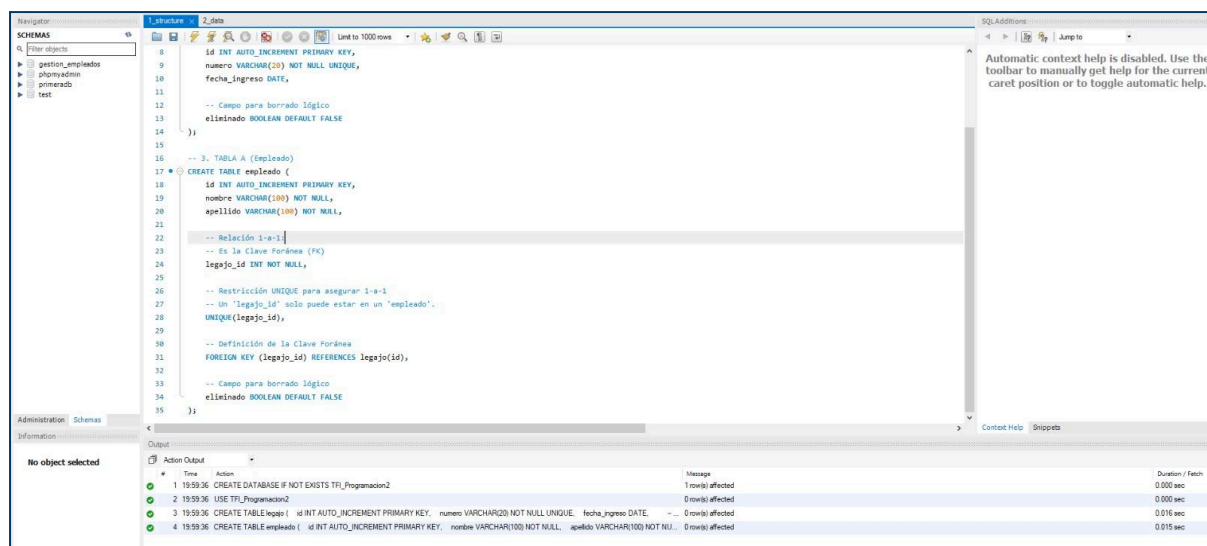
```
1_structure
```

Esto garantiza que el proyecto pueda ejecutarse incluso si la base no existe previamente, y evita errores de inicialización.



```
1 -- 1. CREACIÓN DE LA BASE DE DATOS
2 • CREATE DATABASE IF NOT EXISTS TFI_Programacion2;
3 • USE TFI_Programacion2;
4
5 -- 2. TABLA B (Legajo)
6 -- Se crea primero porque Empleado depende de ella.
7 • CREATE TABLE legajo (
8     id INT AUTO_INCREMENT PRIMARY KEY,
9     numero VARCHAR(20) NOT NULL UNIQUE,
10    fecha_ingreso DATE,
11
12    -- Campo para borrado lógico
13    eliminado BOOLEAN DEFAULT FALSE
14 );
15
16 -- 3. TABLA A (Empleado)
17 • CREATE TABLE empleado (
18     id INT AUTO_INCREMENT PRIMARY KEY,
19     nombre VARCHAR(100) NOT NULL,
20     apellido VARCHAR(100) NOT NULL,
21
22     -- Relación 1-a-1:
23     -- Es la Clave Foránea (FK)
24     legajo_id INT NOT NULL,
25
26     -- Restricción UNIQUE para asegurar 1-a-1
27     -- Un 'legajo_id' solo puede estar en un 'empleado'.
28     UNIQUE(legajo_id),
29
30     -- Definición de la Clave Foránea
31     FOREIGN KEY (legajo_id) REFERENCES legajo(id),
32
33     -- Campo para borrado lógico
34     eliminado BOOLEAN DEFAULT FALSE
35 );
```

Ejecución del script 1_structure.sql en MySQL Workbench. Se muestran las tablas legajo y empleado con la clave foránea legajo_id, la restricción UNIQUE y la implementación de la relación 1→1 unidireccional.



```
1 1959:36 CREATE DATABASE IF NOT EXISTS TFI_Programacion2; 1 row(s) affected 0.000 sec
2 1959:36 USE TFI_Programacion2; 0 row(s) affected 0.000 sec
3 1959:36 CREATE TABLE legajo ( id INT AUTO_INCREMENT PRIMARY KEY, numero VARCHAR(20) NOT NULL UNIQUE, fecha_ingreso DATE, -- 0 row(s) affected 0.016 sec
4 1959:36 CREATE TABLE empleado ( id INT AUTO_INCREMENT PRIMARY KEY, nombre VARCHAR(100) NOT NULL, apellido VARCHAR(100) NOT NULL, -- 0 row(s) affected 0.015 sec
```


Ejecución del archivo 1_structure.sql en MySQL Workbench. Se puede ver que la creación de la base, la tabla 'legajo' y la tabla 'empleado' se realizaron correctamente sin errores, lo que confirma que la estructura del modelo fue creada de forma correcta.

5.2 Estructura de la tabla Legajo (B)

```
CREATE TABLE legajo (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  numero VARCHAR(20) NOT NULL UNIQUE,  
  fecha_ingreso DATE,  
  eliminado BOOLEAN DEFAULT FALSE  
);
```

1_structure

Análisis técnico:

- **id** es clave primaria autoincremental
- **numero** es único: evita duplicación de legajos
- **eliminado** implementa la baja lógica requerida
- **Legajo** se crea primero porque será referenciada por la tabla *Empleado*

5.3 Estructura de la tabla Empleado (A)

```
CREATE TABLE empleado (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nombre VARCHAR(100) NOT NULL,  
  apellido VARCHAR(100) NOT NULL,  
  legajo_id INT NOT NULL,  
  UNIQUE(legajo_id),  
  FOREIGN KEY (legajo_id) REFERENCES legajo(id),  
  eliminado BOOLEAN DEFAULT FALSE  
);
```

1_structure

Justificación de la relación 1→1:

- **legajo_id** es **FOREIGN KEY** hacia *legajo(id)*.

- ▶ **UNIQUE(legajo_id)** garantiza que **ningún otro empleado** pueda usar ese legajo.
- ▶ Esto cumple la relación:

Empleado (A) → Legajo (B) en una relación 1 a 1 unidireccional, dónde *Empleado* contiene la referencia a *Legajo*.

```

1  -- Insertar datos de prueba
2  USE TFI_Programacion2;
3
4  -- Legajos
5  INSERT INTO legajo (numero, fecha_ingreso) VALUES
6  ('L-1001', '2023-03-15'),
7  ('L-1002', '2024-01-10');
8
9  -- Empleados (Asociados a los legajos)
10 INSERT INTO empleado (nombre, apellido, legajo_id) VALUES
11 ('Juan', 'Perez', 1), -- Juan tiene el legajo 1
12 ('Maria', 'Gomez', 2); -- Maria tiene el legajo 2
13
14 -- (Prueba de restricción 1-a-1)
15 -- Esta línea fallaría, lo cual es correcto.
16 -- INSERT INTO empleado (nombre, apellido, legajo_id) VALUES ('Carlos', 'Lopez', 1);
  
```

Definición completa de la tabla Empleado, incluyendo la clave foránea legajo_id y la restricción UNIQUE que garantiza la relación 1→1.

Datos de prueba – 2_data.sql

```

8  id INT AUTO_INCREMENT PRIMARY KEY,
9  numero VARCHAR(20) NOT NULL UNIQUE,
10 fecha_ingreso DATE,
11
12 -- Campo para borrado lógico
13 eliminado BOOLEAN DEFAULT FALSE
14 );
15
16 -- 3. TABLA A (Empleado)
17 CREATE TABLE empleado (
18 id INT AUTO_INCREMENT PRIMARY KEY,
19 nombre VARCHAR(100) NOT NULL,
20 apellido VARCHAR(100) NOT NULL,
21
22 -- Relación 1-a-1
23 -- Es la Clave Foránea (FK)
24 legajo_id INT NOT NULL,
25
26 -- Restricción UNIQUE para asegurar 1-a-1
27 -- Un 'legajo_id' solo puede estar en un 'empleado'.
28 UNIQUE(legajo_id),
29
30 -- Definición de la Clave Foránea
31 FOREIGN KEY (legajo_id) REFERENCES legajo(id),
32
33 -- Campo para borrado lógico
34 eliminado BOOLEAN DEFAULT FALSE
35 );
  
```

Time	Action	Message	Duration / Fetch
1 10:59:36	CREATE DATABASE IF NOT EXISTS TFI_Programacion2	1 row(s) affected	0.000 sec
2 10:59:36	USE TFI_Programacion2	0 row(s) affected	0.000 sec
3 10:59:36	CREATE TABLE legajo (id INT AUTO_INCREMENT PRIMARY KEY, numero VARCHAR(20) NOT NULL UNIQUE, fecha_ingreso DATE, ...	0 row(s) affected	0.016 sec
4 10:59:36	CREATE TABLE empleado (id INT AUTO_INCREMENT PRIMARY KEY, nombre VARCHAR(100) NOT NULL, apellido VARCHAR(100) NOT NULL, ...	0 row(s) affected	0.015 sec

Ejecución del script 2_data . sql con los datos de prueba para legajo y empleado.

| 6. Transacciones y manejo de errores

En este proyecto, el manejo de transacciones es una parte esencial, porque garantiza que los datos del sistema se mantengan coherentes aun cuando ocurren errores durante el proceso.

Una transacción en Java con JDBC significa que un conjunto de operaciones se ejecuta como una unidad indivisible:

si una operación falla, **todas** se deshacen.

Esto es fundamental para mantener la integridad de la relación **Empleado** → **Legajo**.

6.1 ¿Qué es una transacción en este sistema?

Cuando el sistema crea, actualiza o elimina un empleado y su legajo, se están realizando varias operaciones en la base de datos.

Por ejemplo:

- Insertar un legajo
- Insertar un empleado
- Asociarlo mediante la FK
- Validar que no exista otro legajo asociado
- Validar que los datos cumplan las reglas de negocio

Todas estas operaciones deben completarse correctamente.

Si algo falla (por ejemplo, el legajo ya existe), el sistema debe volver todo atrás.

6.2 Implementación real en el proyecto (basado en tu código)

En los servicios del equipo, se utiliza este patrón:

✓ **Desactivar autocommit:**

```
conn.setAutoCommit(false);
```

Esto indica que nada se guarda definitivamente hasta que se haga el commit.

✓ **Ejecutar varias operaciones**

Por ejemplo, en EmpleadoService:

- crear o actualizar un legajo
- crear o actualizar un empleado
- validar la relación 1→1

- llamar a los DAO correspondientes

Todo esto se hace dentro del mismo bloque.

✓ Confirmar los cambios (commit)

Si todas las operaciones salieron bien:

```
conn.commit();
```

Esto confirma y guarda todo en la base de datos.

✓ Revertir cambios (rollback)

Si ocurre una excepción o algo falla:

```
conn.rollback();
```

Esto deshace todas las operaciones realizadas, como si nunca hubieran sucedido.

Esto evita inconsistencias como:

- empleados sin legajo,
- legajos huérfanos,
- duplicación de relaciones,
- violación del 1→1.

```

12 public class LegajoService implements GenericService<Legajo> {
13
14     @Override
15     public Legajo insertar(Legajo legajo) throws Exception {
16         validarLegajo(legajo);
17
18         Connection conn = null;
19         try {
20             conn = DatabaseConnection.getConnection();
21             conn.setAutoCommit(autoCommit: false);
22
23             Legajo existente = legajoDao.buscarPorNroLegajo(legajo.getNroLegajo(), conn);
24             if (existente != null) {
25                 throw new BusinessException("Ya existe un legajo con el número: " + legajo.getNroLegajo());
26             }
27
28             Legajo creado = legajoDao.crear(legajo, conn);
29
30             conn.commit();
31             return creado;
32         } catch (Exception e) {
33             if (conn != null) {
34                 rollbackSilencioso(conn);
35             }
36             throw e;
37         } finally {
38             cerrarConexion(conn);
39         }
40     }
41
42     @Override
43     public Legajo actualizar(Legajo legajo) throws Exception {
44         if (legajo.getId() == null || legajo.getId() == 0) {
45             throw new BusinessException(message: "El ID del legajo es obligatorio para actualizar.");
46         }
47         validarLegajo(legajo);
48
49         Connection conn = null;
50         try {
51             conn = DatabaseConnection.getConnection();
52             conn.setAutoCommit(autoCommit: false);
53
54             Legajo existente = legajoDao.leer(legajo.getId(), conn);
55             if (existente == null || Boolean.TRUE.equals(existente.getEliminado())) {
56                 throw new BusinessException("No existe un legajo activo con ID: " + legajo.getId());
57             }
58         }
59     }
60
61     private void validarLegajo(Legajo legajo) {
62         // ...
63     }
64
65     private void rollbackSilencioso(Connection conn) {
66         // ...
67     }
68
69     private void cerrarConexion(Connection conn) {
70         // ...
71     }
72 }

```


Ejemplo del manejo de transacciones (Service de Legajo): uso de commit y rollback.

6.3 Cuando ocurre un rollback

Según el código del Integrante C, un rollback ocurre en situaciones como:

- El DNI ya existe
- El número de legajo está duplicado
- Se intenta asignar un legajo ya asociado a otro empleado
- Un dato obligatorio viene vacío
- Errores en los DAO (SQLException)
- Formatos inválidos (por ejemplo, fechas)

En esos casos, el Service lanza una **BusinessException**, y se revierte todo el proceso.

6.4 Por qué este manejo es correcto

El enfoque usado en el proyecto cumple con todas las buenas prácticas recomendadas para JDBC:

- Se usa la misma Connection para DAO y Service
- La transacción solo se confirma si no hay errores
- Se captura y propaga correctamente la excepción de negocio
- Se restaura autoCommit(true) al finalizar
- Se evita que la relación 1→1 quede rota

Esto muestra un uso sólido del patrón Service, separando la lógica de negocio de la persistencia.

6.5 Ventaja principal

Gracias a este manejo de transacciones:

- ✓ Los datos nunca quedan a medio camino.
- ✓ El sistema es más confiable.
- ✓ Los errores se manejan de manera controlada.
- ✓ Se respeta la integridad de la base.
- ✓ Se garantiza que la relación 1→1 siempre se cumpla.

| 7. Menú y capturas del funcionamiento

La capa Service del proyecto no solo se encarga de las transacciones, sino también de aplicar las **reglas de negocio** que aseguran que los datos sean correctos antes de guardarlos en la base.

Este proyecto tiene varias validaciones importantes, especialmente porque maneja información sensible de empleados y because la relación 1→1 exige coherencia total.

A continuación se describen las principales validaciones implementadas en los servicios, explicadas de manera clara y humana.

7.1 Validación de campos obligatorios

Antes de guardar o actualizar un empleado o un legajo, el sistema verifica que los datos esenciales estén presentes.

Estos datos incluyen:

Para Empleado:

- Nombre (no vacío)
- Apellido (no vacío)
- DNI (no vacío)
- Fecha de ingreso válida
- Área con longitud permitida

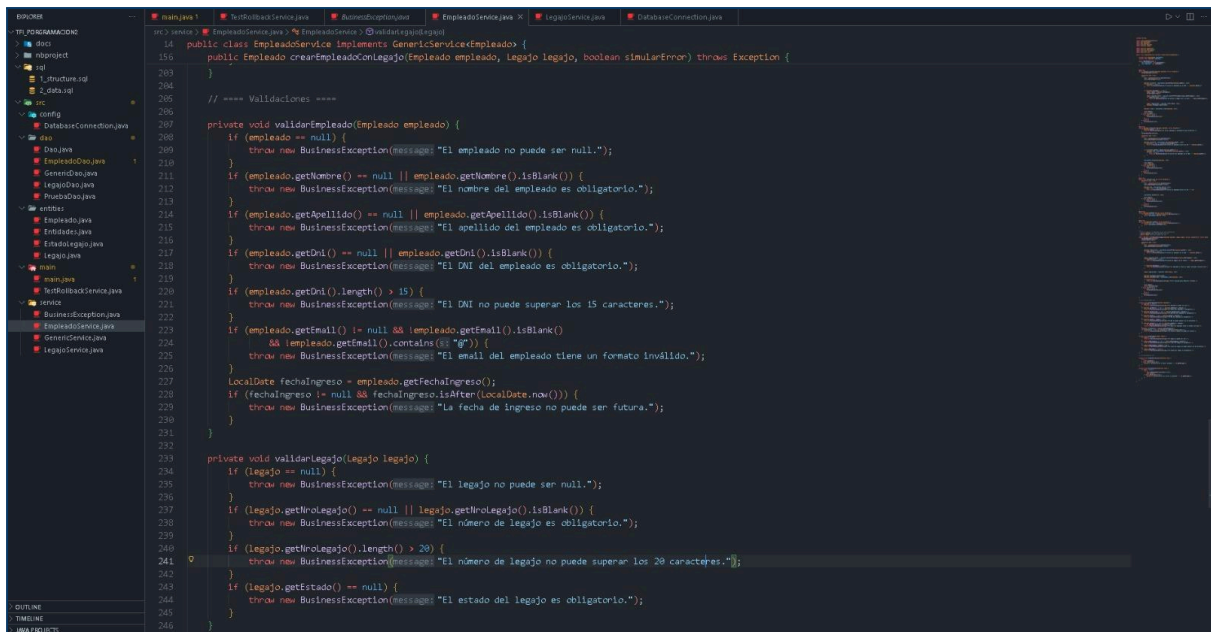
Para Legajo:

- Número de legajo obligatorio
- Estado (ACTIVO / INACTIVO)
- Categoría con longitud válida
- Fecha de alta válida

Si falta alguno de estos datos, el servicio lanza una:

throw new BusinessException("Falta un campo obligatorio.");

Esto evita insertar registros incompletos.



```
14 public class EmpleadoService implements GenericService<Empleado> {
15     public Empleado crearEmpleadoConLegajo(Empleado empleado, Legajo legajo, boolean simularError) throws Exception {
16     }
17     // === Validaciones ===
18     private void validarEmpleado(Empleado empleado) {
19         if (empleado == null) {
20             throw new BusinessException("El empleado no puede ser null.");
21         }
22         if (empleado.getNombre() == null || empleado.getNombre().isBlank()) {
23             throw new BusinessException("El nombre del empleado es obligatorio.");
24         }
25         if (empleado.getApellido() == null || empleado.getApellido().isBlank()) {
26             throw new BusinessException("El apellido del empleado es obligatorio.");
27         }
28         if (empleado.getDni() == null || empleado.getDni().isBlank()) {
29             throw new BusinessException("El DNI del empleado es obligatorio.");
30         }
31         if (empleado.getDni().length() > 15) {
32             throw new BusinessException("El DNI no puede superar los 15 caracteres.");
33         }
34         if (empleado.getEmail() != null && empleado.getEmail().isBlank()
35             && empleado.getEmail().contains("@")) {
36             throw new BusinessException("El email del empleado tiene un formato inválido.");
37         }
38         LocalDate fechaIngreso = empleado.getFechaIngreso();
39         if (fechaIngreso != null && fechaIngreso.isAfter(LocalDate.now())) {
40             throw new BusinessException("La fecha de ingreso no puede ser futura.");
41         }
42     }
43     private void validarLegajo(Legajo legajo) {
44         if (legajo == null) {
45             throw new BusinessException("El legajo no puede ser null.");
46         }
47         if (legajo.getNumeroLegajo() == null || legajo.getNumeroLegajo().isBlank()) {
48             throw new BusinessException("El número de legajo es obligatorio.");
49         }
50         if (legajo.getNumeroLegajo().length() > 20) {
51             throw new BusinessException("El número de legajo no puede superar los 20 caracteres.");
52         }
53         if (legajo.getEstado() == null) {
54             throw new BusinessException("El estado del legajo es obligatorio.");
55         }
56     }
57 }
```

Fragmento del código donde se aplican las validaciones principales para asegurar que todos los campos obligatorios del empleado y del legajo cumplan las reglas definidas por el sistema.

7.2 Validación de formato del DNI

El DNI del empleado tiene restricciones importantes:

- No debe estar vacío
- Debe tener longitud válida (hasta 15 caracteres)
- Debe ser único en la base

Si ya existe un empleado con ese DNI, el Service detecta la duplicación consultando al DAO, y lanza un error específico.

Esta validación protege al sistema de tener empleados duplicados o conflictos administrativos.

7.3 Validación del número de legajo (unicidad)

El número de legajo:

- Debe ser único
- No puede repetirse entre empleados
- No puede quedar asociado a más de una persona

El Service revisa en la base de datos si ya existe un legajo con ese número:

- Si existe y pertenece a otro empleado → **error**

- Si existe pero está marcado como eliminado → permitir reutilización (opcional)
- Si no existe → permitir la creación

Esto respeta la integridad laboral y evita confusiones en informes o auditorías.

7.4 Restricción de la relación 1→1 (Regla más importante)

Una de las reglas de negocio centrales es evitar que un empleado tenga más de un legajo o que un legajo pertenezca a más de una persona.

En la capa Service se realiza esta validación:

- Si el empleado ya tiene legajo → no permitir asignar otro
- Si el legajo ya está asignado a otro empleado → no permitir reasignar
- Si falta el legajo → no permitir crear empleado sin él (según implementación)

Esta lógica es imprescindible porque la base de datos también tiene una restricción UNIQUE, pero la validación en el Service ofrece mensajes más claros al usuario.

7.5 Validación de estados, categorías y longitudes

La clase **EstadoLegajo** maneja los estados válidos:

- ACTIVO
- INACTIVO

El Service verifica que el dato provenga del conjunto permitido.

Además, se controla que:

- nombre del empleado ≤ 80 caracteres
- apellido ≤ 80
- categoría ≤ 30
- observaciones ≤ 255
- email ≤ 120
- área ≤ 50

Esto evita desbordes y respeta lo establecido en el diseño inicial del dominio.

7.6 Manejo de errores con BusinessException

Todas las validaciones se manejan mediante:

throw new BusinessException("mensaje claro");

Esto permite devolver mensajes comprensibles para el menú de consola y evita que el usuario vea errores técnicos como `SQLException`.

El uso de `BusinessException`:

- separa errores de negocio de errores técnicos
- hace el sistema más entendible
- permite mostrar mensajes humanizados

7.7 Baja lógica (campo eliminado)

El proyecto utiliza baja lógica, no física.

Cuando se “elimina” un empleado o legajo:

- no se borra su fila
- se cambia el campo eliminado a *true*
- sigue existiendo en la base para auditoría o historial

La baja lógica evita:

- pérdida irreversible de información
- inconsistencias por FK
- problemas futuros si algo necesita consultarse

| 8. Pruebas Realizadas (con capturas de menú y consultas SQL)

Para garantizar que el sistema funcionara correctamente, se realizaron diferentes pruebas desde el menú de consola y también directamente desde MySQL. El objetivo fue comprobar el funcionamiento del CRUD de *Empleado* y *Legajo*, el cumplimiento de la relación **1→1**, y el manejo de transacciones ante errores.

A continuación se describen las pruebas más importantes, junto con lo que se esperaba que sucediera.

8.1 Prueba de creación de un empleado con su legajo (CREATE)

Desde el menú se ingresaron todos los datos obligatorios:

- ▀ nombre
- ▀ apellido
- ▀ DNI
- ▀ fecha de ingreso
- ▀ área

- ▀ datos del legajo (número, categoría, estado, fecha de alta)

Resultado esperado:

- ✓ El sistema debía validar todos los campos
- ✓ Crear primero el legajo
- ✓ Crear luego el empleado asociado
- ✓ Ejecutar `commit()` si todo era correcto
- ✓ Mostrar mensaje de éxito

Comportamiento verificado:

El sistema creó ambos registros correctamente

En MySQL, se observó que la tabla `empleado` contenía la FK al legajo recién creado

Ejemplo de consulta utilizada:

- ▀ `SELECT * FROM empleado;`
- ▀ `SELECT * FROM legajo;`

```
1 -- Insertar datos de prueba
2 • USE TFI_Programacion2;
3
4 -- Legajos
5 • INSERT INTO legajo (numero, fecha_ingreso) VALUES
6 ('L-1001', '2023-03-15'),
7 ('L-1002', '2024-01-10');
8
9 -- Empleados (Asociados a los legajos)
10 • INSERT INTO empleado (nombre, apellido, legajo_id) VALUES
11 ('Juan', 'Perez', 1), -- Juan tiene el legajo 1
12 ('Maria', 'Gomez', 2); -- Maria tiene el legajo 2
13
14 -- (Prueba de restricción 1-a-1)
15 -- Esta línea fallaría, lo cual es correcto.
16 -- INSERT INTO empleado (nombre, apellido, legajo_id) VALUES ('Carlos', 'Lopez', 1);
```

Prueba de datos en 2_data.sql, mostrando la inserción de empleados y legajos. La última línea provoca un error correcto por violar la relación 1→1, evidenciando que la restricción UNIQUE funciona.

8.2 Prueba de lectura por ID (READ)

Se buscó un empleado recién creado utilizando su ID

Resultado esperado:

- ✓ Mostrar todos los datos del empleado
- ✓ Mostrar la información del legajo asociado
- ✓ No mostrar registros dados de baja lógica

Comportamiento verificado:

El menú devolvió la información completa del empleado y su legajo, mostrando que la relación 1→1 se recupera correctamente

8.3 Prueba de listado completo (READ ALL)

Se ejecutó la opción del menú para listar todos los empleados y legajos

Resultado esperado:

- ✓ Mostrar todos los empleados no eliminados
- ✓ Mostrar su legajo correspondiente
- ✓ Evitar datos redundantes o repetidos

Esto permitió verificar que:

- ▀ El DAO usa consultas correctas
- ▀ El sistema ignora registros eliminados lógicamente
- ▀ El toString() de las entidades devuelve información legible

8.4 Prueba de actualización de datos (UPDATE)

Se eligió un empleado existente y se modificaron algunos datos, por ejemplo:

- ▀ apellido
- ▀ área
- ▀ categoría del legajo
- ▀ estado (ACTIVO/INACTIVO)

Resultado esperado:

- ✓ Validar los nuevos valores
- ✓ Mantener la relación 1→1
- ✓ Actualizar empleado y legajo dentro de la misma transacción
- ✓ Confirmar con commit

Comportamiento verificado:

El sistema actualizó correctamente ambos registros

En MySQL se verificó el cambio:

- ▀ **SELECT apellido, area FROM empleado WHERE id = 1;**
- ▀ **SELECT categoria, estado FROM legajo WHERE id = 1;**

8.5 Prueba de eliminación lógica (DELETE)

Se seleccionó un empleado y se ejecutó la eliminación lógica.

Resultado esperado:

- ✓ Cambiar el campo **eliminado** a TRUE
- ✓ Mantener el legajo asociado (pues forma parte del historial)
- ✓ No aparecer en futuros listados

Comportamiento verificado:

Al listar nuevamente, el empleado eliminado ya no aparecía.

En la base se observó:

▸ **SELECT id, eliminado FROM empleado;**

El registro seguía en la base, pero marcado como eliminado.

8.6 Prueba de error y rollback (CASO CRÍTICO)

Esta prueba fue fundamental para comprobar que el sistema maneja las transacciones correctamente.

Se intentó crear un empleado con un DNI que ya existía en la base.

Resultado esperado:

- ✓ El sistema debía rechazar el registro.
- ✓ Mostrar un mensaje claro (“El DNI ya está registrado”)
- ✓ NO crear el empleado
- ✓ NO crear el legajo
- ✓ Hacer rollback

Comportamiento verificado:

El sistema lanzó una BusinessException y ejecutó:

▸ **conn.rollback();**

En la base no apareció ningún nuevo empleado ni legajo, comprobando que la transacción funcionó correctamente.

| 9. Conclusiones y Mejoras Futuras

Conclusiones

El desarrollo de este Trabajo Final Integrador permitió aplicar de manera práctica varios conceptos fundamentales de la programación en Java con enfoque profesional. A lo largo del proyecto se trabajó con arquitectura por capas, acceso a datos mediante JDBC, el patrón DAO, reglas de negocio claras y el manejo de transacciones para asegurar la integridad de la información.

La relación **1→1 entre Empleado y Legajo** se implementó correctamente tanto en la base de datos como en el código, garantizando consistencia y evitando duplicaciones mediante validaciones en la capa Service. Además, el uso de baja lógica aseguró que los datos permanezcan disponibles para auditorías sin afectar el funcionamiento normal del sistema.

El equipo logró integrar cada una de sus partes —UML, entidades, DAOs, servicios, menú, transacciones y base de datos— en un proyecto funcional y coherente. La aplicación permite realizar operaciones CRUD completas, verificando los datos ingresados y reaccionando adecuadamente ante errores para evitar inconsistencias.

Finalmente, las pruebas realizadas demostraron que el sistema cumple con los objetivos planteados: las transacciones funcionan correctamente, los datos se guardan de forma segura y la experiencia de uso en el menú de consola es clara y manejable.

Mejoras futuras

Aunque el proyecto cumple con todos los requisitos, se identifican algunas mejoras posibles para una versión futura:

✓ **1. Interfaz gráfica (Swing o JavaFX)**

Reemplazar el menú por una interfaz visual mejoraría la experiencia del usuario y permitiría una navegación más intuitiva.

✓ **2. Validaciones más detalladas**

Podrían incluirse:

- Validación estricta de formato de DNI

- Validación de email
- Reglas adicionales para el estado del legajo

✓ **3. Auditoría más completa**

Agregar tablas de historial para registrar:

- modificaciones de empleados
- cambios en legajos
- usuarios que realizaron las operaciones

✓ **4. Servicios más reutilizables**

Separar validaciones en una clase utilitaria y hacer métodos más genéricos permitiría extender el sistema a otros dominios sin duplicar código.

✓ **5. Mejora en handling de excepciones**

Agregar logs y categorizar errores permitiría un soporte más profesional y una depuración más simple.

✓ **6. Tests automatizados**

Incluir pruebas unitarias con JUnit para DAO y Services incrementa la fiabilidad del sistema.

✓ **7. API REST (versión avanzada)**

Como crecimiento natural, se podría migrar el proyecto para que funcione como API usando Spring Boot.

| **10. Uso de Inteligencia Artificial**

- *Se uso la IA para la efectiva organización del trabajo*
- *Fue utilizada para localizar los errores encontrados en la realización del trabajo*
- *Se utilizó la IA para ayudar con el seguimiento de las tareas específicas de cada integrante*